

01_phase3_overview

Video Script: Phase 3 Overview

Duration: 8 minutes **Difficulty:** Beginner **Goal:** Introduce Phase 3 features and benefits

[0:00-0:30] Introduction

[ON SCREEN: HelixCode logo, "Phase 3 Overview"]

SCRIPT: "Welcome to HelixCode Phase 3! I'm excited to show you the powerful new features that take AI-assisted development to the next level. In this video, we'll explore what's new, why it matters, and how these features work together to supercharge your development workflow."

[Visual: Animated transition showing Phases 1, 2, 3]

[0:30-1:30] What is Phase 3?

[ON SCREEN: Title - "Phase 3: Advanced AI Development Features"]

SCRIPT: "Phase 3 introduces five major features designed to make AI-assisted development more powerful, persistent, and productive."

First, we have Session Management - think of this as organizing your work into focused development sessions, whether you're planning, building, testing, or debugging.

Second, the Memory System - your AI assistant now remembers the entire conversation history across sessions, maintaining context as you work.

Third, State Persistence - all your sessions, conversations, and context are automatically saved and restored, so you never lose your progress.

Fourth, the Template System - reusable templates for code generation, prompts, and workflows that speed up common tasks.

And finally, these all integrate seamlessly to create a cohesive development experience."

[Visual: Show each system icon appearing on screen as mentioned]

[1:30-3:00] The Five Core Features

[ON SCREEN: Split screen showing each feature]

SCRIPT: “Let’s dive a bit deeper into each feature.

Session Management lets you organize work into sessions with different modes. Planning mode for design, Building mode for implementation, Testing mode for quality assurance, Refactoring for improvements, Debugging for fixes, and Deployment mode for releases. Each session tracks its status, duration, and associated project.

The Memory System maintains conversation history with intelligent management. It tracks messages, counts tokens, and can search through past conversations. You can set limits, trim old messages, and export important conversations.

State Persistence ensures nothing is ever lost. It auto-saves your work at configurable intervals, supports multiple formats like JSON and compressed GZIP, and includes backup and restore capabilities. Atomic writes guarantee data integrity.

The Template System provides six types of templates: Code templates for generation, Prompt templates for AI interactions, Workflow templates for processes, Documentation templates, Email templates, and Custom templates for anything else. Variables with placeholders make them flexible and reusable.

Integration - and here’s the magic - all these systems work together. Your sessions contain conversations from the memory system. Templates generate content for conversations. State persistence saves everything. It’s a complete ecosystem.”

[Visual: Animation showing data flow between systems]

[3:00-4:30] Real-World Benefits

[ON SCREEN: Title - “Why Phase 3 Matters”]

SCRIPT: “So why does this matter for your development workflow?

Context Continuity: Imagine you’re debugging a complex issue. You can start a debugging session, have a detailed conversation with the AI assistant, take a break, come back tomorrow, and everything is exactly where you left it. The

session status, conversation history, even your position in the codebase - all preserved.

Knowledge Reuse: Build a library of templates for your common patterns. Code review prompts, bug fix templates, feature implementation workflows - create once, use everywhere.

Team Collaboration: Export sessions and templates to share with team members. Import their templates into your workflow. Everyone benefits from collective knowledge.

Reliability: With auto-save and state persistence, you never lose work. Power outage? System crash? No problem - restore from the last save point.

Productivity: Spend less time on boilerplate and more time on creative problem-solving. Templates and sessions structure your work, the memory system maintains context, and persistence ensures continuity.”

[Visual: Show real developers working with HelixCode, highlighting moments of efficiency]

[4:30-6:30] Quick Demo Preview

[ON SCREEN: Live demo environment]

SCRIPT: “Let me show you a quick example of how this works in practice.

I’m starting a new feature development session...”

[Demo Actions]:

```
// Create a session
session := sessionMgr.Create("implement-auth",
                             session.ModeBuilding, "auth-service")
sessionMgr.Start(session.ID)

// Start a conversation
conv := memoryMgr.CreateConversation("Auth Implementation")
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(
    "Help me implement JWT authentication",
))

// Use a template
prompt, _ := templateMgr.RenderByName("Code Review",
    map[string]interface{}{
        "language": "Go",
```

```
    "code": authCode,  
  })  
  
  // Auto-save kicks in  
  // [Show auto-save indicator]
```

SCRIPT CONTINUES: “Notice how the session is tracking our work, the conversation maintains context, and everything is being automatically saved in the background.

Now let me simulate closing and reopening...

[Demo: Close and restore]

And we’re back! The session status is preserved, our conversation history is intact, and we can pick up exactly where we left off. That’s the power of Phase 3.”

[Visual: Show seamless restoration]

[6:30-7:30] Use Cases

[ON SCREEN: Title - “Perfect For”]

SCRIPT: “Phase 3 is perfect for several scenarios:

Long-running projects where you need to maintain context over days or weeks.

Complex debugging where you need to track investigation history and findings.

Code review workflows using template-based review prompts and session tracking.

Team environments where sharing sessions and templates improves consistency.

Learning and experimentation where you want to preserve your exploration journey.

Production development where reliability and data integrity are critical.”

[Visual: Icons or short clips for each use case]

[7:30-8:00] What’s Next

[ON SCREEN: Course module list]

SCRIPT: “In the upcoming videos, we’ll dive deep into each feature: - Getting started with setup and configuration - Session management in detail - Building with the memory system - State persistence best practices - Creating and using templates - And finally, advanced integration patterns

By the end of this course, you’ll master all Phase 3 features and be able to build powerful, persistent AI-assisted workflows.

Let’s get started!”

[ON SCREEN: “Next: Getting Started” with play button]

Supplementary Materials

Code Examples (shown in video)

```
// Session creation
session := sessionMgr.Create("feature-name",
    session.ModeBuilding, "project-id")
sessionMgr.Start(session.ID)

// Conversation
conv := memoryMgr.CreateConversation("Feature Discussion")
memoryMgr.AddMessage(conv.ID, memory.NewUserMessage("Help me
    implement..."))

// Template rendering
result, _ := templateMgr.RenderByName("Function",
    map[string]interface{}{
        "function_name": "processData",
        "parameters": "data []byte",
        "return_type": "error",
        "body": "return nil",
    })

// State save
store.Save()
```

Key Takeaways

1. Phase 3 adds 5 integrated features for advanced AI development
2. Session Management organizes work by mode and project
3. Memory System maintains conversation context
4. State Persistence ensures nothing is lost

5. Template System enables reusable patterns
6. All features integrate seamlessly

Resources

- Phase 3 Completion Summary: `/docs/PHASE_3_COMPLETION_SUMMARY.md`
- Integration Guide: `/docs/PHASE_3_INTEGRATION_GUIDE.md`
- API Documentation: `/docs/api/`

Quiz Questions

1. What are the 6 session modes in Phase 3?
2. Name the 5 core features of Phase 3
3. What does the Memory System track?
4. What formats does State Persistence support?
5. How many template types are available?